Doubly Linked List Traversal (Forward and Backward)

```
#include <stdio.h>
#include <stdlib.h>
// Structure of Node
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};
// Function to create a new node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}
// Function to insert at end
void insertEnd(struct Node** head, int value) {
    struct Node* newNode = createNode(value);

    // If list is empty
    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;
```

```
    // Traverse to last node
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}
// Forward Traversal
void traverseForward(struct Node* head) {
    struct Node* temp = head;
    printf("Forward Traversal: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
// Backward Traversal
void traverseBackward(struct Node* head) {
    struct Node* temp = head;
    // Move to last node
    while (temp->next != NULL) {
        temp = temp->next;
    }
    printf("Backward Traversal: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->prev;
```

```
    }
    printf("\n");
}

// Main Function
int main() {
    struct Node* head = NULL;
    // Inserting elements
    insertEnd(&head, 10);
    insertEnd(&head, 20);
    insertEnd(&head, 30);
    insertEnd(&head, 40);

    // Traversals
    traverseForward(head);
    traverseBackward(head);
    return 0;
}
```